

**DUALSHOCK 4 DROP**

**LAB 1**

**SECTION:**

**H**

**SUBMITTED BY:**

**Simon Samuel**

**SUBMISSION DATE:**

**10/20/2025**

## **Problem**

The objective of lab 5 was to make a program that could detect and measure the speed of a DualShock 4 controller while it is falling. We wrote the program in C, and the lab was split into 2 parts. Part one was identifying when the controller was dropped, when it finished falling, how much time it took to fall, as well as how many meters the controller fell, all while ignoring air resistance. During part two, we had to make a program that did include air resistance, which overall made the results more accurate to what is really happening when the controller is dropped.

## **Analysis**

The inputs came from the accelerometer, which provided time and acceleration values in the x, y, and z directions. Using these values, as well as the magnitude formula, the acceleration was able to be calculated. The change in acceleration was used to identify when the controller had been dropped and was falling. The program identified when the controller was freefalling as well as when the controller hit the ground. What was output was a string of text which was required by the lab, but using the calculations it helped output the distance traveled as well as the seconds it took to fall. Distance traveled was calculated using the formula,  $x = \frac{1}{2} g t^2$ . But in part two, where it was required to calculate for air resistance, I had to use two more formula:  $v_i = v_{i-1} + g(1 - |a|)(t_i - t_{i-1})$ ,  $x_i = x_{i-1} + v_i(t_i - t_{i-1})$ . Absolute value of "a" represents the magnitude of acceleration, each loop, after small time intervals, updated the values for position and velocity. This allowed the program to integrate the motion over time. It was assumed the controller started at rest, and that it was moving fast enough to capture accurate measurements. Adding this produces smaller distance values, which shows how air resistance reduces the distance an object travel.

## **Design**

When I first started the program, I targeted the simplest stuff first. I started by telling the program to print the preliminary messages before the drop. Then it would run through a loop, where it would detect if the acceleration had changed, if it didn't, then it would print a "." every second or so. After it detected the fall, it then printed another message "Help me! I'm falling!" the next loop began by printing a "!" then it would use tolerance to see if the controller had stopped or not. After that the program calculated the total fall time and distance, using the formula for constant acceleration, then it printed the results in a sentence. For part 2, where I needed to include air resistance, I added new variables for velocity and position, the loop was then modified so it would update the values after the readings. Using the difference in time, the program found the new velocity and position values that changed during each iteration. After calculating this, the program printed the original distance as well as the distance considering air resistance. I also used functions as well as constants to make readability easier.

## **Testing**

To test the program, I did some small drops, from about a meter high. This helped to make sure my time and distance values were accurate. There was some small variation throughout the tests which also helped me to determine good tolerance values for the controller that I had used. We were also provided with some data samples to run the code with; it was nice having these values since the other controllers can be finnick with the results that they provide. The files simulated a drop from the third floor of the engineering building, similarly we did our own drops from the third floor which helped to verify results of our code even more.

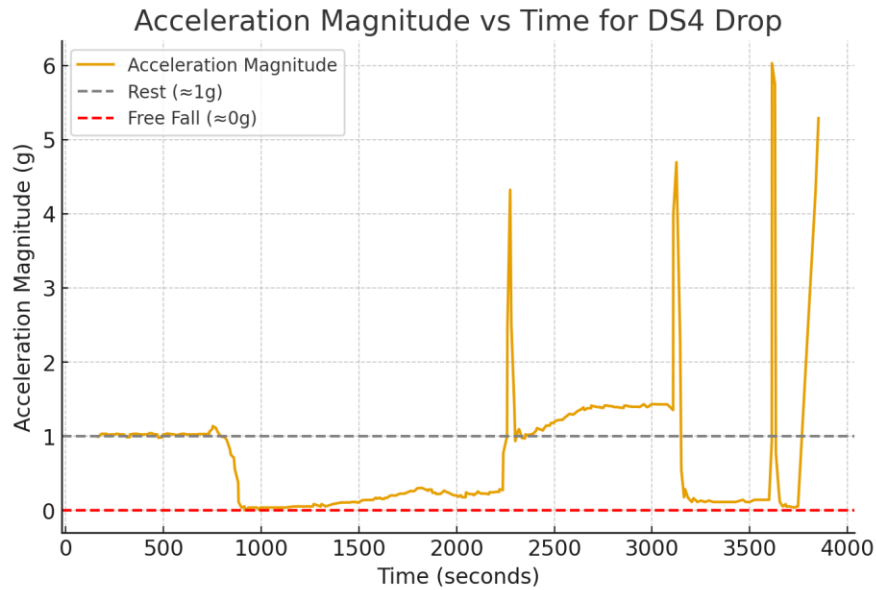
## **Comments**

This lab taught me about how real-world physics can be modeled using a program. In part one it was a struggle to properly create the loops but once I managed one set of loops it made creating the next set a whole lot easier. In part 2 figuring out what I needed to change was a little challenging, especially since I needed to create a whole new set of variables for it too. During testing, finding the proper tolerances was a challenge, going to the TLA and TA office hours, using the controllers in this space didn't always provide consistent results. Sometimes because of the differences between controller hardware, software, and calibration, one tolerance didn't work for all controllers. This was by far the most challenging lab, and finishing definitely felt accomplished. I like how this lab incorporated nearly everything that we have learned so far, as the other labs seemed to focus on a particular subject.

## **Questions and Experiments**

Part 1:

1. Results were consistent once I got the tolerance down, all I can think of for any of the variation that I saw is just human error.
2. Using sample data 1, the distance from the top floor to the ground is 9.023 meters.



3. The tolerances I decided to use for both free fall and landing is 0.5. The reason for this value was simply through trial and error, it was a value that was closest to getting accurate and consistent results, if I went less it barely registered and if I went to high it registered to much skipping important loops. Though it did vary by controller, which is why my results using other controllers sometimes weren't accurate.

#### Part 2:

1. The difference was anywhere between %5 to %8 of the distance without air resistance for both in the lab and the 3<sup>rd</sup> floor.
2. According to my code it was about 6 meters, but when I spoke with the TA that didn't seem to be accurate. Which seemed to be an issue with my tolerances.
3. The hardest part of part two was implementing the formulas into my existing code and creating new variables for the new information.

## Implementation

Part 1:

```
/*-----
-           Prototypes
-----*/

double mag(double x, double y, double z);
int close_to(double tolerance, double point, double value);

/*-----
-           Implementation
-----*/

#include <stdio.h>

#include <stdlib.h>

#include <math.h>

#include <time.h>


#define G 9.8           // gravity constant
#define WAIT_DOTS 10    // print one '.' every this many lines
#define FALL_EXCLAMATIONS 10 // print one '!' every this many lines
#define FREEFALL_TOLERANCE 0.5 // tolerance for detecting ~0g
#define HIT_TOLERANCE 0.5    // tolerance for detecting hit (~1g)


int main(void) {
    char firstname[] = "Simon";
    char lastname[] = "Samuel";
    char netid[] = "simonsj1";
```

```

double t, ax, ay, az;

double a_mag;

int count = 0;

int falling = 0;

double start_time = 0.0, end_time = 0.0;


printf("%s %s\n", firstname, lastname);

printf("%s\n\n", netid);

printf("Ok, I'm now receiving data.\n");

printf("I'm Waiting ");


while (scanf("%lf,%lf,%lf,%lf", &t, &ax, &ay, &az) == 4) {

    a_mag = mag(ax, ay, az);


    // If not falling yet
    if (!falling) {

        if (close_to(FREEFALL_TOLERANCE, 0.0, a_mag)) {

            // start of freefall

            falling = 1;

            start_time = t;

            printf("\n\nHelp me! I'm falling");

            count = 0;

        } else {

            // still waiting

```

```

        if (count % WAIT_DOTS == 0)

            printf(".");

            fflush(stdout);

        }

    }

    else { // falling == 1

        if (close_to(HIT_TOLERANCE, 1.0, a_mag)) {

            // end of fall

            end_time = t;

            double fall_time = end_time - start_time;

            fall_time = fall_time / 1000;

            double distance = 0.5 * G * fall_time * fall_time;

            printf("\n\nOuch! I fell %.3f meters in %.3f seconds.\n", distance, fall_time);

            break;

        } else {

            if (count % FALL_EXCLAMATIONS == 0)

                printf("!");

            fflush(stdout);

        }

    }

    count++;

}

return 0;

}

```

```
double mag(double ax, double ay, double az) {
    return sqrt(ax*ax + ay*ay + az*az);
}
```

```
int close_to(double tolerance, double point, double value) {
    return fabs(point - value) < tolerance;
}
```

Part 2:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
#include <time.h>
```

```
#define G 9.8          // gravity constant
```

```
#define WAIT_DOTS 10    // print one '.' every this many lines
```

```
#define FALL_EXCLAMATIONS 10 // print one '!' every this many lines
```

```
#define FREEFALL_TOLERANCE 0.4 // tolerance for detecting ~0g
```

```
#define HIT_TOLERANCE 0.4    // tolerance for detecting hit (~1g)
```

```
// Function prototypes
```

```
double mag(double x, double y, double z);
```

```
int close_to(double tolerance, double point, double value);
```

```
int main(void) {
```



```

char firstname[] = "Simon";

char lastname[] = "Samuel";

char netid[] = "simonsj1";


double t, ax, ay, az;

double a_mag;

int count = 0;

int falling = 0;

double start_time = 0.0, end_time = 0.0;


// Variables for air resistance model

double t_prev = 0.0, v_prev = 0.0, x_prev = 0.0;


printf("%s %s\n", firstname, lastname);

printf("%s\n\n", netid);

printf("Ok, I'm now receiving data.\n");

printf("I'm Waiting ");


while (scanf("%lf,%lf,%lf,%lf", &t, &ax, &ay, &az) == 4) {

    a_mag = mag(ax, ay, az);


    // If not yet falling

    if (!falling) {

        if (close_to(FREEFALL_TOLERANCE, 0.0, a_mag)) {

            falling = 1;

            start_time = t;

```

```

    t_prev = t;

    printf("\n\nHelp me! I'm falling");

    count = 0;

} else {

    if (count % WAIT_DOTS == 0)

        printf(".");

        fflush(stdout);

    }

}

else { // falling

    double dt = (t - t_prev) / 1000.0; // convert ms to s


    // air resistance model

    double v = v_prev + G * (1 - a_mag) * dt;

    double x = x_prev + v * dt;


    // Check for landing

    if (close_to(HIT_TOLERANCE, 1.0, a_mag)) {

        end_time = t;

        double fall_time = (end_time - start_time) / 1000.0;

        double distance_no_resistance = 0.5 * G * fall_time * fall_time;


        printf("\n\nOuch! I fell %.3f meters in %.3f seconds.\n", distance_no_resistance,
fall_time);

        printf("Compensating for air resistance, the fall was %.3f meters.\n", x);

```

```

        double percent_diff = ((distance_no_resistance - x) / distance_no_resistance) *
100.0;

        printf("This is %.1f%% less than computed before.\n", percent_diff);


        break;
    } else {
        if (count % FALL_EXCLAMATIONS == 0)
            printf("!");
        fflush(stdout);
    }


    v_prev = v;
    x_prev = x;
    t_prev = t;
}


    count++;
}


return 0;
}


double mag(double ax, double ay, double az) {
    return sqrt(ax * ax + ay * ay + az * az);
}

```

```
int close_to(double tolerance, double point, double value) {  
    return fabs(point - value) < tolerance;  
}
```